

SHORT-TERM DEVELOPMENT PLAN

Robot Dog Browser Environment

Feature roadmap for the current dog and current browser-based interface

Scope: improve operation, safety, calibration, telemetry, control, debugging and gait development without turning the project into a generic multi-robot platform yet.

Robot Dog Software Roadmap

13 August 2026

Document map

This document is intended to be used as a build roadmap: start at the top, implement the high-value foundations first, and leave the speculative features until the core robot workflow is reliable.

- 1. Scope and objective** What the browser version should become and what it should deliberately avoid.
- 2. Priority stack** A build order that separates must-have foundations from optional polish.
- 3. Connection and transport** A reliable connection state, Bluetooth/controller handling and command transport.
- 4. Safety and state control** Arm/disarm, e-stop, watchdogs and explicit robot state.
- 5. Telemetry** Useful live measurements rather than decorative dashboards.
- 6. Calibration** Per-joint setup that removes hard-coded offsets from firmware.
- 7. Controller and manual control** Gamepad mapping, deadzones, test views and input diagnostics.
- 8. IK and gait tuning** Expose the variables that are currently painful to change in code.
- 9. Diagnostics and logs** Make failures localisable instead of guessing where the command chain stopped.
- 10. UI structure** A desktop-first layout that keeps the 3D view central.
- 11. Implementation roadmap** Suggested phases and a definition of done.
- 12. Deferred features** Useful ideas that should wait until the core workflow is stable.

1. Scope and objective

The short-term goal is not to build a universal robotics application. It is to make the existing browser environment the best possible engineering interface for the current dog.

The current version already has the hardest foundation: a working interactive environment built around the dog. The highest-value work now is to make that environment reliable enough to operate, tune and debug the physical robot without repeatedly changing firmware or opening separate tools.

For this phase, it is acceptable for the software to know exactly what robot it is controlling: four legs, three joints per leg, the current ESP32/PCA9685 control stack, the existing IMU and power sensing, and the current controller scheme. Generic abstractions can be introduced later where they are cheap, but they should not slow down getting the present dog working well.

Design rule

If a feature directly reduces the time between “I changed something” and “I understand what the robot did”, it belongs in this version. If it primarily exists to support hypothetical future robots, it belongs in the desktop-platform roadmap.

2. Priority stack

Priority	Feature group	Why it matters
P0	Connection state + heartbeat	Everything else depends on knowing whether commands and telemetry are actually moving.
P0	Arm / disarm / e-stop	Prevents accidental motion and defines a safe state for development.
P0	Telemetry essentials	Battery, current, IMU, packet timing and robot state make behaviour observable.
P1	Calibration screen	Moves servo offsets, directions and limits out of source code and into a repeatable workflow.
P1	Controller diagnostics	Shows whether an input problem is controller, browser, transport, gait or servo output.
P1	IK + gait tuning	Lets you tune geometry and locomotion without recompiling firmware.
P2	Logs and plots	Makes intermittent behaviour and tuning sessions reviewable.
P2	Profiles / presets	Lets you switch between safe, test and performance configurations quickly.

Priority	Feature group	Why it matters
P3	Camera / advanced visualisation	Useful later, but not needed to establish a reliable control loop.

3. Connection and transport

3.1 Connection manager

The interface should always have one authoritative connection state. Avoid scattered “connected” booleans in different components. A central connection manager should own discovery, connection, reconnect, latency and the last time a valid packet was received.

```

ConnectionState
  disconnected
  connecting
  connected
  degraded
  reconnecting
  fault

```

- Persistent connection indicator visible on every screen.
- Robot address / transport shown next to the status.
- Round-trip latency or command-to-acknowledgement timing.
- Last telemetry packet age.
- Automatic reconnect with clear UI feedback rather than silently failing.
- Explicit disconnect button for bench work and fault recovery.

3.2 Bluetooth and controller connectivity

Bluetooth is worth adding where it solves a real connection problem, but it should be treated as one transport option rather than the entire architecture. For the current browser build, support depends on the browser and operating system, so the software should expose the connection capability cleanly and fail gracefully if that transport is unavailable.

- Bluetooth gamepad support through the browser Gamepad API where available.
- Optional Bluetooth link to robot-side hardware only if it is technically useful versus the existing Wi-Fi/serial path.
- Device selection screen that names the active controller and shows reconnect state.
- Do not tie gait logic directly to Bluetooth events; convert all inputs into a common control-command object first.

3.3 Command pipeline

```

Controller / Keyboard / UI
  |
  v
Input normalisation
deadzones + scaling + mode
  |
  v
Robot command
velocity / yaw / height
  |
  v
Transport
Wi-Fi / serial / other
  |
  v
ESP32

```

A single command representation is important even in the dog-specific browser version. It makes debugging far easier and gives the later desktop application a clean point to attach additional transports.

4. Safety and state control

The software should make it impossible to be unsure whether the robot is allowed to move.

Control	Behaviour	Implementation note
Disarmed	No locomotion commands are accepted.	Default state after connect, reload or communication fault.
Arm	Requires deliberate user action.	Prefer hold-to-arm or a short confirmation gesture.
E-stop	Immediately commands a safe halt.	Large persistent control when armed.
Heartbeat timeout	Stops motion if command packets disappear.	Must be enforced on the robot, not only in the browser.
Fault state	Blocks re-arming until the fault is acknowledged.	Show the reason: low voltage, transport loss, invalid command, etc.

Important

The browser can provide a good operator interface, but the final communication-loss fail-safe must live on the robot controller as well. If the browser crashes or the network disappears, the ESP32 still needs a bounded timeout to stop gait output.

5. Telemetry

Telemetry should answer three questions: Is the robot healthy? Is the command loop working? What is the robot physically doing? Start with measurements that help answer those questions, then add higher-detail data only when the hardware can supply it reliably.

5.1 Core dashboard

Metric	Display	Why
Battery voltage	V + warning threshold	Basic power-health indicator.
Current	A	Shows load spikes, stalls and gait demand.
Calculated power	W	Useful for comparing gait efficiency and transient load.
Roll / pitch / yaw	degrees + small attitude graphic	Shows IMU state and body attitude.

Metric	Display	Why
Command latency	ms	Makes network/controller lag visible.
Packet rate	Hz	Confirms the transport is updating at the expected rate.
Dropped / stale packets	count or %	Identifies communication quality.
ESP32 loop rate	Hz	Shows whether firmware load is affecting control.
Robot state	armed / disarmed / fault / gait	Prevents ambiguous operation.
Uptime	hh:mm:ss	Useful when correlating faults during testing.

5.2 Joint-level telemetry

Even if the current servos cannot report position or temperature, the app can still show the commanded joint values. That is enough to identify whether the gait/IK stack is producing sane outputs.

- Commanded angle for all 12 joints.
- Servo output / PWM value after calibration mapping.
- Joint limit warning when a target is clipped.
- Foot target X/Y/Z for each leg.
- IK validity flag and unreachable-target warning.
- Later: actual servo position, current and temperature if future actuators expose them.

5.3 Plots

Do not make plots the default dashboard. Put them in a dedicated “scope” view where a few signals can be selected and graphed over the last 10-60 seconds. High-value traces are current, voltage, roll/pitch, commanded foot Z, joint angles and command latency.

6. Calibration

Calibration is one of the highest-value additions because it turns a mechanical build variation into stored configuration rather than source-code edits.

```

Front Left > Hip

Neutral offset    +3.2 deg
Direction        Inverted
Minimum          -42 deg
Maximum          +46 deg
Servo channel     0

[ -1 ] [ -0.1 ]  LIVE VALUE  [ +0.1 ] [ +1 ]

[ Save joint ]  [ Save leg ]  [ Restore defaults ]

```

- Select the robot visually by clicking a leg/joint in the 3D model.
- Jog one servo at a time at a deliberately limited speed.
- Edit zero offset, output direction and mechanical limits.

- Preview the calibrated neutral pose before saving.
- Save calibration to non-volatile robot storage so a browser reload does not lose it.
- Export/import a calibration backup as JSON.
- Provide a “bench mode” that disables gait generation while calibrating individual joints.

Safety detail

Calibration mode should enforce conservative joint speed and range limits. A typo or stuck key should not command a full-speed sweep into a mechanical stop.

7. Controller and manual control

The controller screen should be both a setup page and a diagnostic instrument. It should expose the raw axis, processed axis and resulting robot command side by side.

```
Left stick X  raw +0.041  -> deadzone 0.000 -> strafe 0.00
Left stick Y  raw -0.734  -> scaled   -0.681 -> forward 0.68
Right stick X raw +0.218  -> scaled   +0.163 -> yaw     0.16

A  Stand      B  Lie
X  Walk       Y  Trot
LB Precision  RB Fast
```

- Controller name and connection status.
- Live raw axis values.
- Deadzone and response-curve adjustment.
- Button mapping editor.
- Per-axis inversion.
- Controller vibration feedback later if useful.
- Keyboard fallback for development.

8. IK and gait tuning

Anything that currently requires changing constants and reflashing firmware is a candidate for a live tuning control, provided it can be bounded safely.

Group	Parameters to expose
Geometry	Upper/lower link length, hip offset, body width/length, nominal body height.
Body pose	Height, roll, pitch, yaw, X/Y translation.
Step geometry	Step length, step height, stance width, touchdown position.
Timing	Cycle time, swing duration, duty factor, phase offsets.
Limits	Maximum velocity, yaw rate, body pose and joint travel.
Gait presets	Walk, trot and a conservative test gait with known-safe defaults.

Use a two-stage apply model for riskier values: edit locally and preview in the 3D model, then explicitly “Apply to robot”. Small command variables such as body height or speed can remain live.

9. Diagnostics and logs

The diagnostic view should make the command chain observable from input to servo output.

```
Controller input      OK  98 Hz
Command generated    OK  50 Hz
Packet transmitted   OK  50 Hz
ESP32 acknowledgement OK  49.9 Hz
Gait target generated OK
IK solved             OK
Joint limit check     OK
Servo command output OK  12 channels
```

- Timestamped event log with severity: info / warning / fault.
- Last transmitted command and last received telemetry packet.
- Packet counters and sequence numbers.
- Clear indication of the first broken stage in the pipeline.
- Download a short diagnostic bundle containing config + recent logs.
- Optional developer console hidden behind an advanced toggle.

10. UI structure

The 3D dog should remain the centre of the product rather than becoming a tiny decoration beside dashboards. Use persistent status at the top, a navigation rail or tabs for tools, and context-sensitive controls around the viewport.

```
+-----+
| DOG V2  | CONNECTED 11 ms 15.8 V | DISARMED | ESTOP |
+-----+
| Control |          3D ROBOT VIEW          | Live state |
| Telemet |                                | Roll  +1.2 |
| Calib   |                                | Pitch -0.8 |
| Gaits   |                                | 7.4 A    |
| Logs    |                                |           |
+-----+
|          | Context controls / controller / timeline |
+-----+
```

- Always-visible: connection, arm state, battery and e-stop.
- Clicking a leg/joint changes the right-side context panel.
- Telemetry summary should not cover the model.
- Advanced settings should be hidden by default to keep normal operation clean.
- Persist user layout/preferences locally, but persist robot-critical calibration on the robot as well.

11. Implementation roadmap

Phase	Build	Exit condition
1 - Observability	Connection manager, heartbeat, status bar, core telemetry, packet counters.	You can tell within seconds whether the browser, link and ESP32 are communicating correctly.

Phase	Build	Exit condition
2 - Safety	Arm/disarm, e-stop UI, command watchdog, explicit fault state.	Reloading, disconnecting or losing packets cannot leave locomotion running.
3 - Calibration	Joint selector, offsets, direction, limits, persistent save/load.	A mechanically reassembled leg can be recalibrated without changing source code.
4 - Control	Controller test page, mappings, deadzones, manual body pose controls.	Every user input can be traced to the generated robot command.
5 - Locomotion tooling	IK inspector, gait parameter panel, presets, safe apply workflow.	Gait tuning can be performed from the UI with visual feedback.
6 - Diagnostics	Signal plots, logs, exportable diagnostic bundle.	Intermittent behaviour can be captured and reviewed after a test.

11.1 Suggested internal data model

```

AppState
|- connection
|- safety
|- controller
|- telemetry
|- robotPose
|- gait
|- calibration
|- diagnostics

RobotCommand
|- velocityX
|- velocityY
|- yawRate
|- bodyHeight
|- bodyRoll / pitch / yaw
|- gaitMode
|- timestamp

```

Even though the app is dog-specific, keeping state in these explicit groups will make the later desktop conversion much easier. The browser UI should not directly manipulate ESP32 packet bytes from individual components.

11.2 Definition of done for the browser version

- Connect to the current dog and show a trustworthy connection/latency state.
- Arm and disarm deliberately, with robot-side communication timeout protection.
- Drive the dog using the existing controller while seeing raw and processed inputs.
- Show battery/current/IMU and control-loop telemetry in real time.
- Select any joint and calibrate offset, direction and limits without code changes.
- Inspect foot targets, IK outputs and commanded joint angles.
- Tune common gait parameters from the browser and save known-good presets.
- Capture useful logs when something stops moving or behaves incorrectly.

12. Deferred features

These are worthwhile, but they should not displace the foundations above.

Feature	Reason to defer
Video streaming	Adds bandwidth and latency concerns before the core control path is stable.
Map / waypoint autonomy	Requires a more mature sensing and autonomy stack.
OTA firmware flashing	Useful, but a bad implementation can brick the controller; add after connection and versioning are reliable.
Full user accounts/cloud sync	Local project first; little value during single-robot development.
Generic robot import	Belongs in the desktop platform rather than the current dog-specific browser build.
Isaac Sim live integration	High value later, but should use the generic application architecture rather than being bolted into this version.
RL policy controls	Meaningful only after the hardware/simulation interface is stable.

Recommended short-term endpoint

The browser version is complete when it feels like a dependable engineering console for this dog: connect, arm, drive, inspect, calibrate, tune, diagnose and stop - all from one place.

At that point, freeze the browser version as the proven reference implementation. The desktop application can then reuse the interface and proven robot-control concepts while replacing hard-coded assumptions with a project-based multi-robot architecture.